Lab Assignment=13.4

Hall.No:2303A51873

Name:R.Vineesha

Batch:14

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

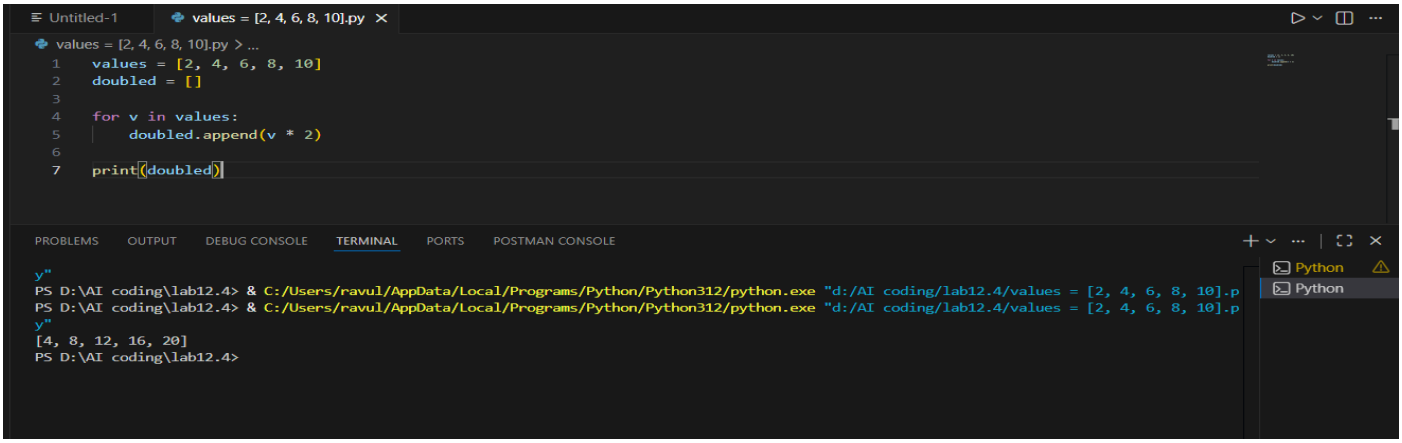
Legacy Code

values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
doubled.append(v * 2)
print(doubled)

Expected Output

[4, 8, 12, 16, 20]

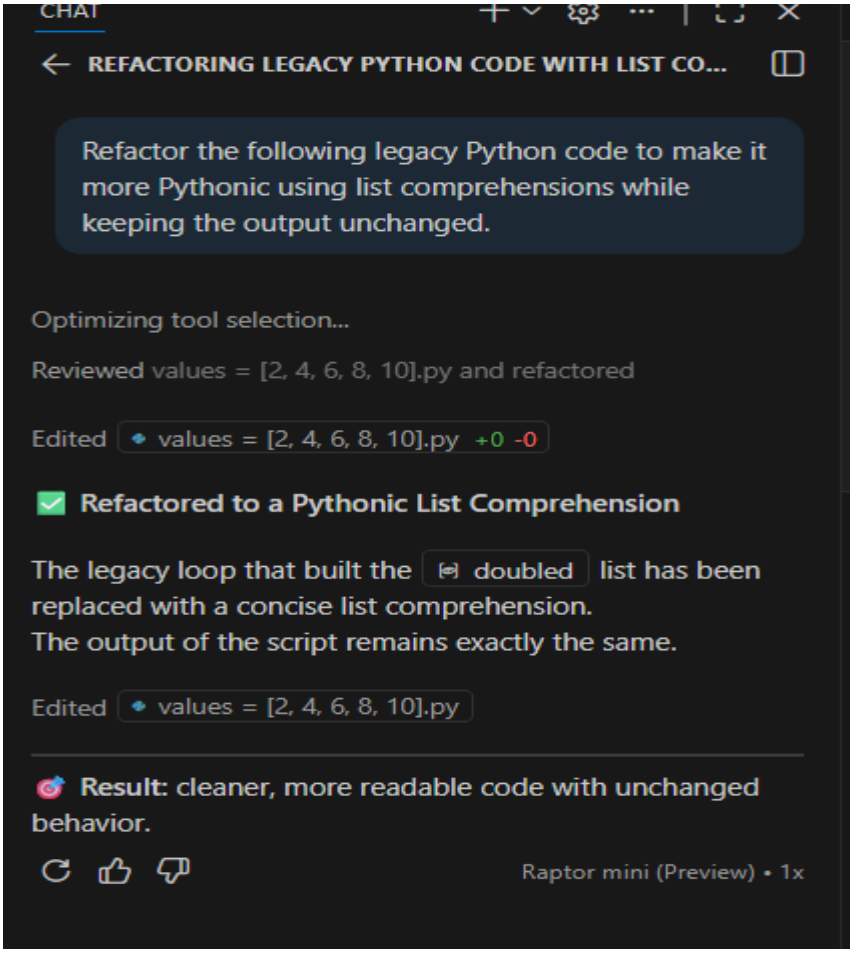
Legacy Code



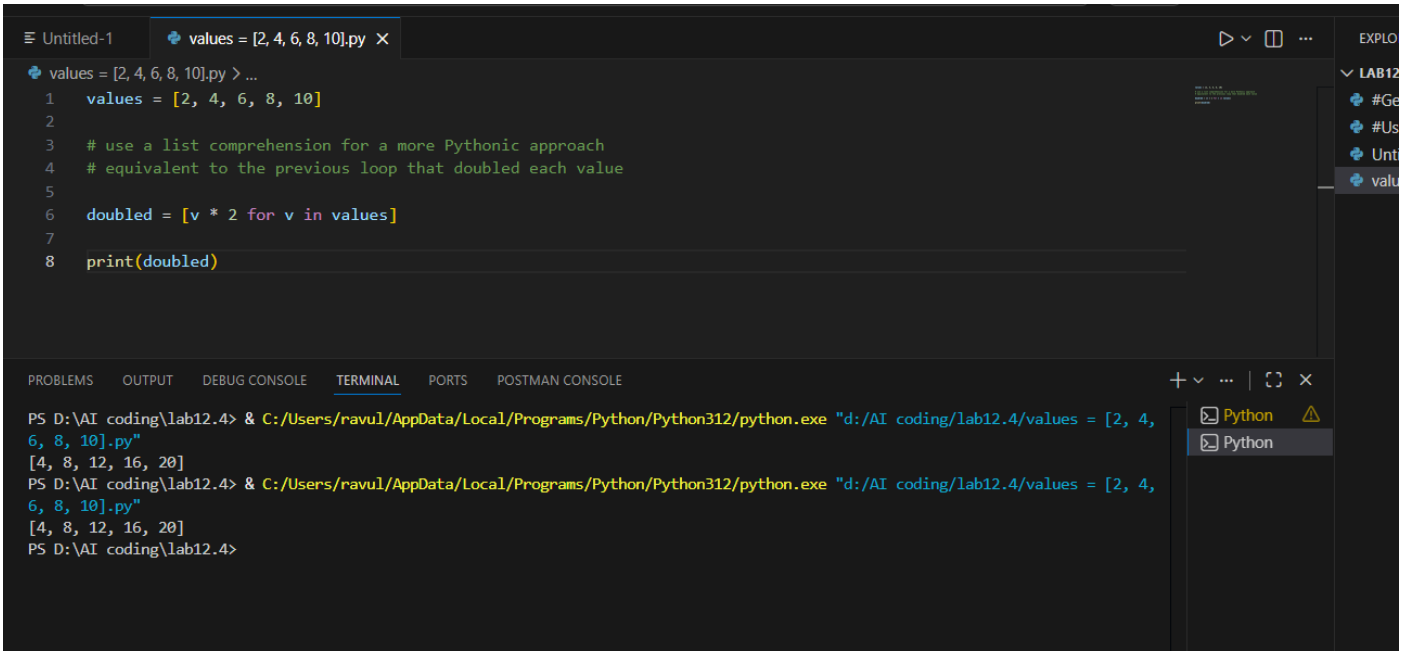
Refactored Code

Prompt

Refactor the following legacy Python code to make it more Pythonic using list comprehensions while keeping the output unchanged.



Code&Output



Observation

1. The legacy Python code used a for loop to iterate through the list and append doubled values to another list.
2. This approach worked correctly but was verbose and less readable.
3. Using an AI-assisted suggestion from tools like ChatGPT or GitHub Copilot, the code was refactored using a list comprehension.
4. The refactored code reduced the number of lines and followed modern Python best practices.
5. The output remained unchanged, confirming that refactoring improved readability without affecting functionality.
6. The refactored version is more concise, efficient, and maintainable.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

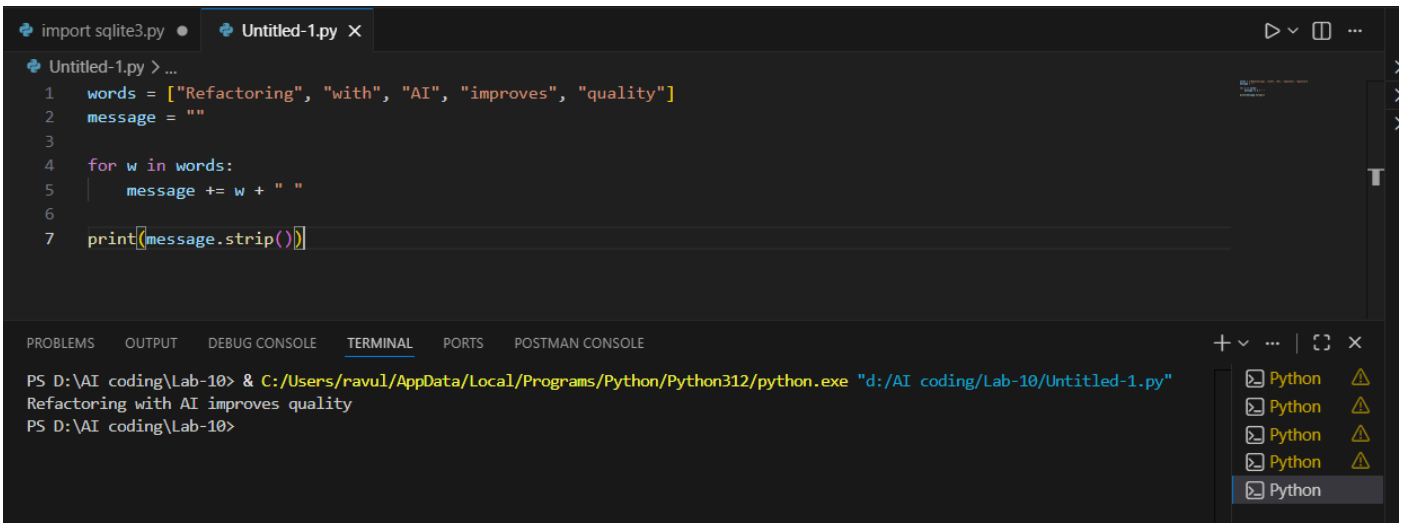
```
words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""

for w in words:
message += w + " "
print(message.strip())
```

Expected Output

Refactoring with AI improves quality

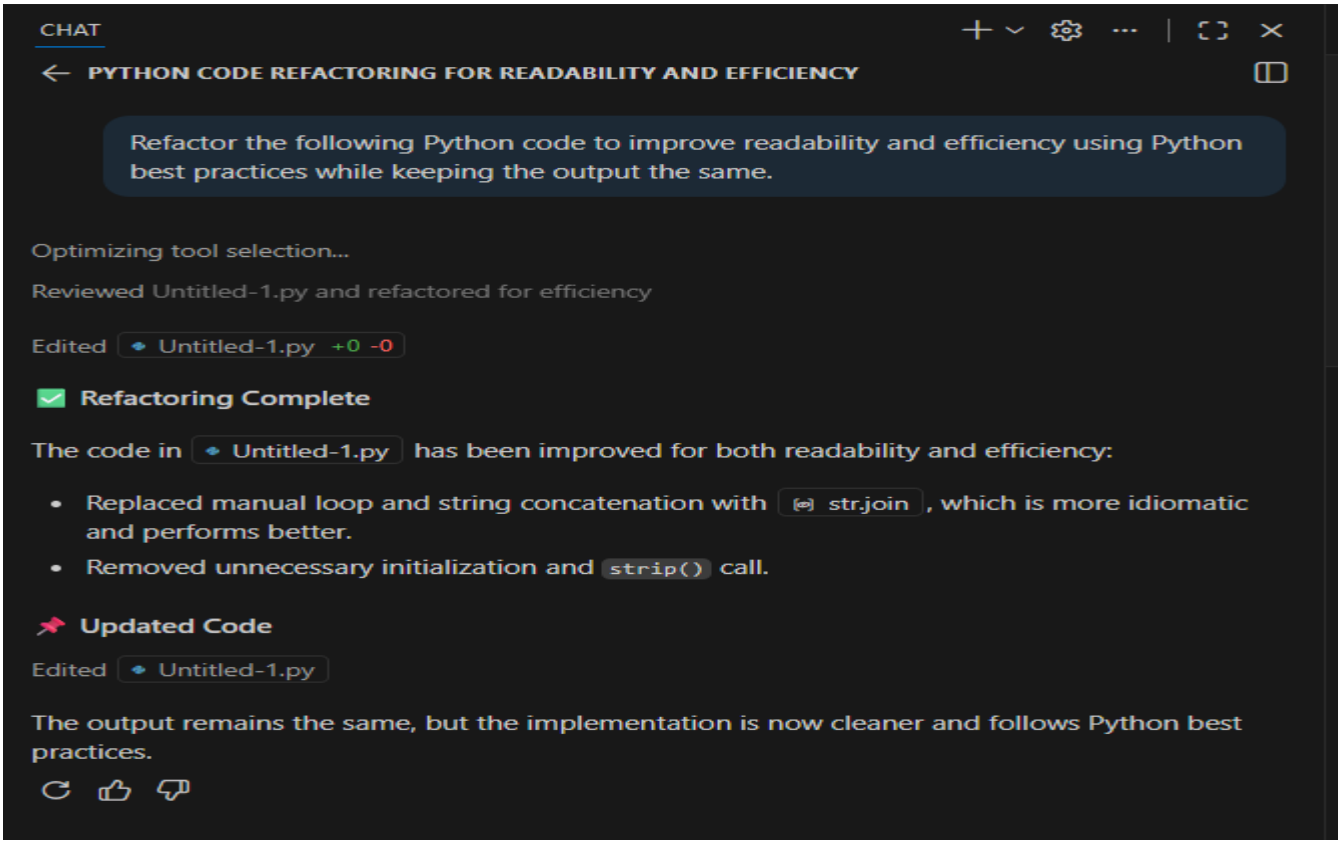
Legacy Code



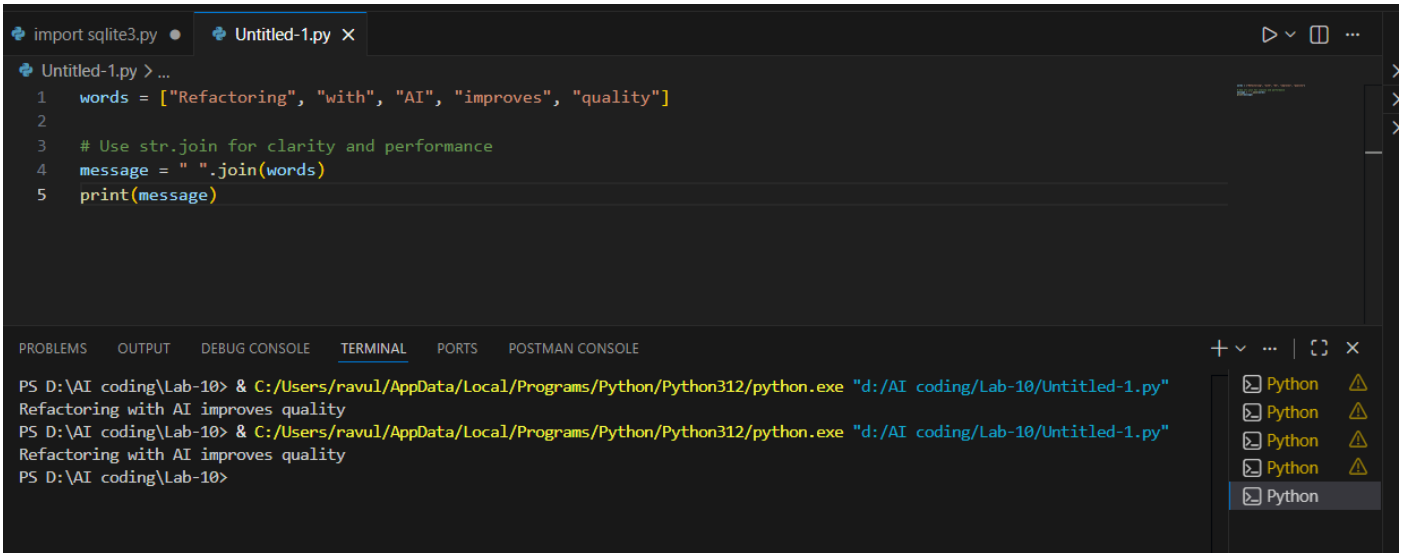
Refactored Code

Prompt

Refactor the following Python code to improve readability and efficiency using Python best practices while keeping the output the same.



Code&Output



Observation

1. The legacy code generated a sentence using repeated string concatenation inside a loop.
2. This approach works but is less efficient and harder to read, especially for large text data.
3. Using an AI suggestion (e.g., from ChatGPT), the code was refactored using Python's join() method.
4. The join() function combines all words in the list into a single string with spaces in a more efficient and readable way.
5. The final output remained the same, confirming that the refactoring improved code quality without changing functionality.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
else:
print("Default timeout used")
```

Expected Output

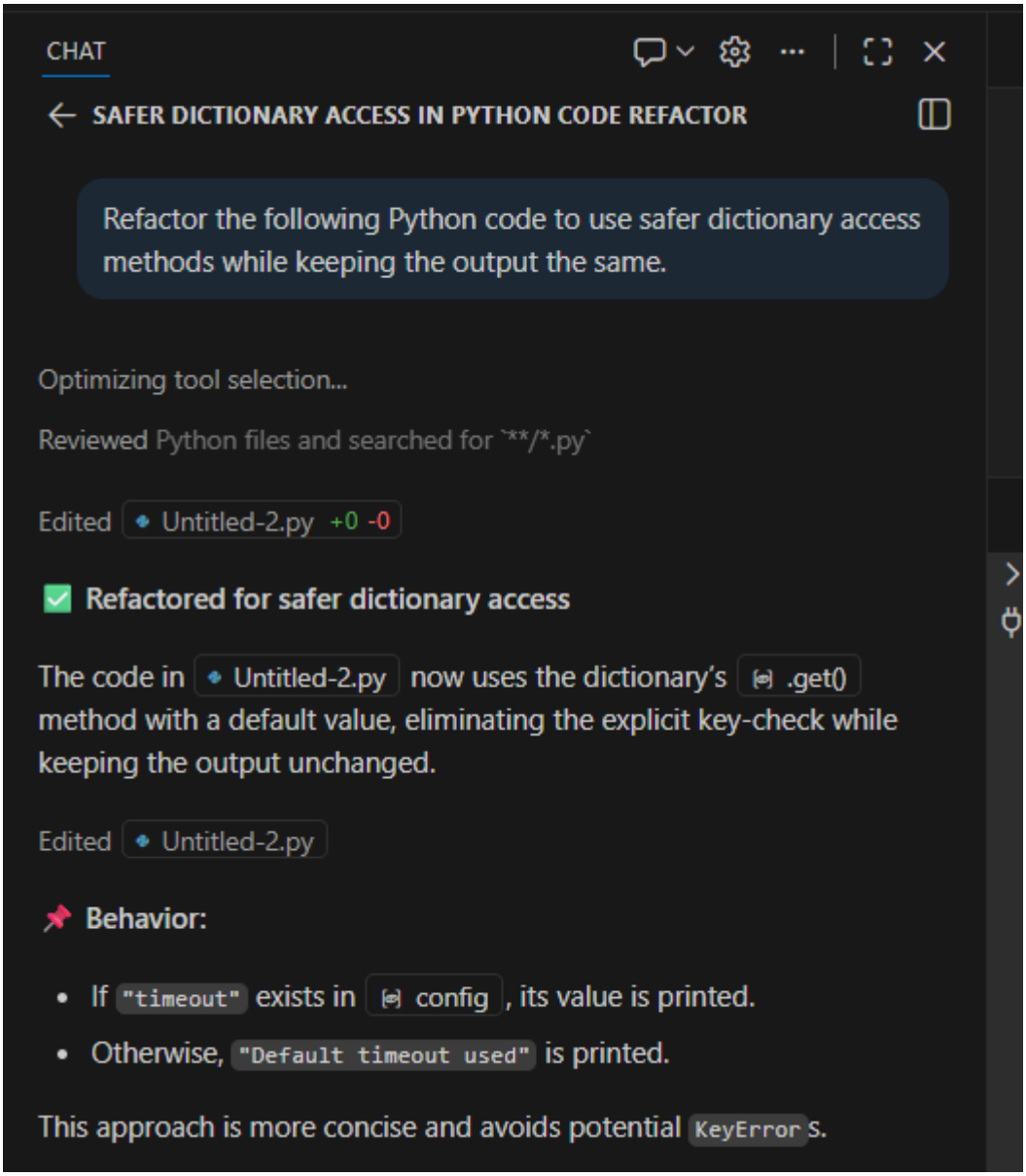
Default timeout used

Legacy Code

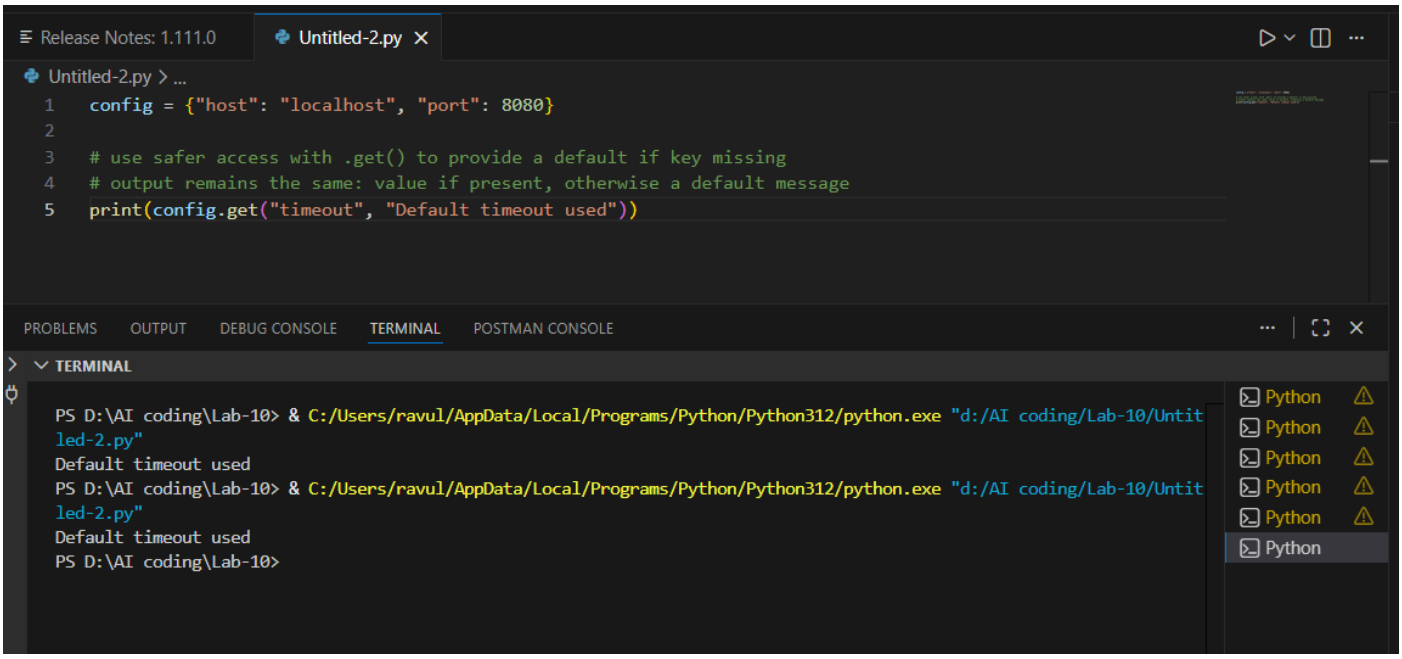
Refactored Code

Prompt

Refactor the following Python code to use safer dictionary access methods while keeping the output the same.



Code&Output



Observation

1. The legacy code checked whether the "timeout" key exists in the dictionary using an if condition.
2. This approach works but requires extra lines of code and manual checking.
3. Using AI suggestions (e.g., from ChatGPT), the code was refactored using the get() method for safer dictionary access.
4. The get() method returns a default value when the key is missing, improving readability and safety.
5. The output remained the same, showing that the refactoring maintained the original behavior while simplifying the code.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
elif action == "subtract":
result = x - y
elif action == "multiply":
result = x * y
elif action == "divide":
result = x / y
else:
result = None
print(result)
```

Expected Output

5.0

Legacy Code

Untitled-1 | values = [2, 4, 6, 8, 10].py

```python
action = "divide"
x, y = 10, 2

if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None

print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE

```
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/values = [2, 4, 6, 8, 10].py"
5.0
PS D:\AI coding\lab12.4>
```

Refactored Code

Prompt

Refactor the following Python code to make it more scalable and readable. Replace multiple if-elif conditions with a cleaner mapping-based approach while keeping the same output

CHAT

REFACTORING PYTHON CODE FOR SCALABILITY AND READABILITY

Refactor the following Python code to make it more scalable and readable. Replace multiple if-elif conditions with a cleaner mapping-based approach while keeping the same output.

Optimizing tool selection...

Reviewed values = [2, 4, 6, 8, 10].py

Edited values = [2, 4, 6, 8, 10].py +0 -0

Refactored for readability and scalability

The original `if`/`elif` chain has been replaced with a clean, dictionary-based approach using function mappings. This makes adding new operations easy and keeps the logic concise.

Changes applied

- Created an `operations` dict mapping action names to lambda functions.
- Replaced conditional block with a lookup and a default behavior.
- Added a simple check to avoid division by zero.

Updated code

Edited values = [2, 4, 6, 8, 10].py

Tip: To support more operations, simply add new entries to the `operations` dict—no need to touch branching logic.

Code&Output

Untitled-1 | values = [2, 4, 6, 8, 10].py

values = [2, 4, 6, 8, 10].py > ...

```python
action = "divide"
x, y = 10, 2

# Use a mapping from operation names to functions for scalability and clarity
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b if b != 0 else None,
}

# Get the appropriate function or default to None
func = operations.get(action)
result = func(x, y) if func else None

print(result)
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | POSTMAN CONSOLE

TERMINAL

```
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/values = [2, 4, 6, 8, 10].py"
5.0
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/values = [2, 4, 6, 8, 10].py"
5.0
PS D:\AI coding\lab12.4>
```

Python
Python
Python

Observation

1. The legacy code used multiple if–elif conditions to perform arithmetic operations based on the selected action.
2. This approach works correctly but becomes difficult to maintain and extend when more operations are added
3. Using AI suggestions from ChatGPT, the code was refactored using a dictionary mapping technique to associate actions with operations.
4. The refactored code is cleaner, more scalable, and easier to modify.
5. The output remained the same (5.0), showing that the refactoring preserved the original functionality while improving code structure.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

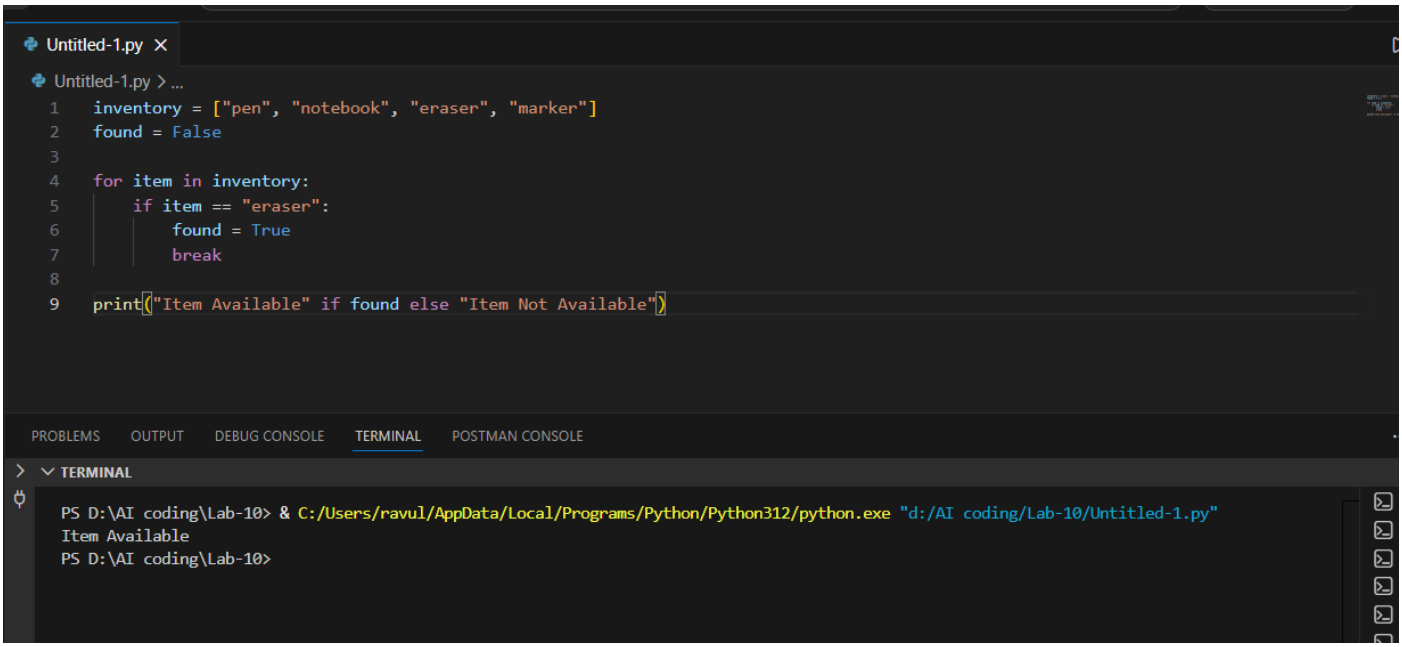
Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

Legacy Code

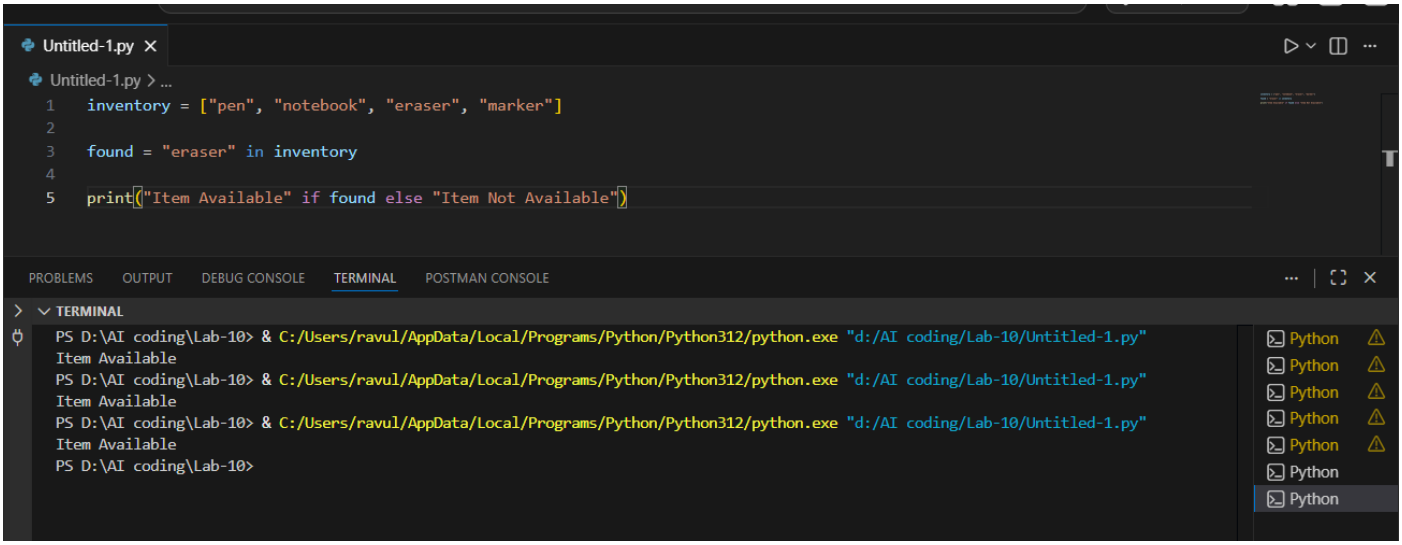


Refactored Code

Prompt

Refactor the following Python code to simplify the search logic using Python best practices while keeping the output the same.

Code&Output



Observation

1. The legacy code used a manual loop to search for an item in the list.
2. This required additional variables and control statements.
3. With AI assistance from ChatGPT, the code was simplified using Python's in membership operator.
4. The refactored code is shorter, clearer, and easier to understand.
5. The output remained unchanged, confirming that the refactoring preserved the original functionality.